King Fahd University of Petroleum & Minerals
Systems & Control Engineering Department

Assessment 2: Differential Flatness Pursuit and Motor Voltage Dynamics

Course: SCE 672 – Advanced Robotics **Instructor:** Prof. Aamer Iqbal Bhatti
Student Name: Raian E Rabbani **Student ID:** 202617880
Target Tag: AprilTag T_3 ($d_0 = 1.50$ m) **Date:** October 2026

Problem 1: Visual Search, In-Place Centering & Standoff Guard

(i) Downward Camera Setup

In our ROS 2 driver node, the robotic arm is initialized to ($X = 105$ mm, $Y = 30$ mm):

```
self.ep_robot.robotic_arm.moveto(x=105, y=30).wait_for_completed()
```

Why this pose works without blocking the Time-of-Flight (ToF) beam:

- **Downward Tilt:** The FPV camera is attached to the upper arm linkage. Retracting the reach to $X = 105$ mm and raising the elevation to $Y = 30$ mm tilts the camera downward by about 22°. This keeps tags on the floor in clear view from 1.8 m down to the 0.20 m standoff.
- **Sensor Beam Clearance:** The front chassis sensor is mounted low on the front bumper. If the arm is lowered or extended forward ($X > 150$ mm, $Y \leq 0$ mm), the gripper hangs in front of the bumper and blocks the beam. Setting ($X = 105$ mm, $Y = 30$ mm) keeps the gripper above the front line-of-sight.

(ii) Search and Centering Finite State Machine

The supervisory state machine uses three primary states:

1. `SEARCHING`: If the target tag is not visible ($\Delta t > 0.45$ s), forward speed is set to zero ($v = 0$) and the robot rotates in place at $\omega_z = \pm 0.30$ rad/s toward the last known direction.
2. `ALIGNING`: Once the tag is spotted, the robot keeps $v = 0$ and turns in place until bearing error is within ±10° ($|\theta_{\text{bearing}}| \leq 0.18$ rad).
3. `CHASING`: Once centered, forward pursuit begins along the generated flat cubic trajectory.

Why in-place alignment prevents trajectory drift and camera loss:

- **Prevents Field-of-View Loss:** The camera has a limited horizontal FOV ($\sim 65°$). Moving forward while bearing error is large sweeps the tag off the side of the frame before the robot finishes turning.
- **Satisfies 1D Boundary Conditions:** The cubic path is planned along the initial robot forward axis with $y(0) = 0$ and $y(T) = 0$. Aligning first ensures $+X_{\text{body}}$ points directly at the tag, preventing lateral overshoot and wheel slippage.

(iii) Standoff Braking and Decoder Hint

- **Active Standoff Brake:** When the estimated distance drops to $d \leq 0.20$ m (20 cm), the node engages an immediate active brake ($v = 0, \omega = 0$) and enters `ARRIVED (20cm)`.
- **Video Decoder Buffer Recycling:** In the camera thread, adding a short pause (`time.sleep(0.010)`) between `cap.grab()` calls gives the FFmpeg decoder time to recycle frames, preventing memory buffer exhaustion (`av_frame_get_buffer` error).

Problem 2: Differential Flatness Trajectory Generation

(i) Differential Flatness Inversion Equations

For a unicycle kinematic model $\dot{x} = v\cos\theta, \dot{y} = v\sin\theta, \dot{\theta} = \omega$, the flat output is the position $z(t) = [x(t), y(t)]^T$. The inputs are recovered algebraically from $z(t)$ and its derivatives:

$$v(t) = \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}, \qquad \omega(t) = \frac{\dot{x}(t)\ddot{y}(t) - \dot{y}(t)\ddot{x}(t)}{\dot{x}(t)^2 + \dot{y}(t)^2} \tag{1}$$

(ii) Cubic Polynomial Matrix Solution

Each flat coordinate $z_j(t)$ ($j \in \{x, y\}$) is a cubic polynomial:

$$z_j(t) = \alpha_0^{[j]} + \alpha_1^{[j]} t + \alpha_2^{[j]} t^2 + \alpha_3^{[j]} t^3 \tag{2}$$

Applying boundary conditions at $t = 0$ and $t = T$ yields the matrix system $\mathbf{M}(T)\boldsymbol{\alpha}^{[j]} = \mathbf{b}^{[j]}$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} \alpha_0^{[j]} \\ \alpha_1^{[j]} \\ \alpha_2^{[j]} \\ \alpha_3^{[j]} \end{bmatrix} = \begin{bmatrix} z_j(0) \\ \dot{z}_j(0) \\ z_j(T) \\ \dot{z}_j(T) \end{bmatrix} \tag{3}$$

In Python, this is solved using `numpy.linalg.solve`:

```
def solve_flat_polynomial(z0, zdot0, zf, zdotf, T):
    M = np.array([
        [1.0, 0.0, 0.0,      0.0],
        [0.0, 1.0, 0.0,      0.0],
        [1.0, T,   T**2,     T**3],
        [0.0, 1.0, 2.0 * T, 3.0 * (T**2)]
    ], dtype=np.float64)
    b = np.array([z0, zdot0, zf, zdotf], dtype=np.float64)
    return np.linalg.solve(M, b)
```

Boundary conditions used:

- **Initial:** $x(0) = 0$, $\dot{x}(0) = 0.06$ m/s; $y(0) = 0$, $\dot{y}(0) = 0$.
- **Final:** $x(T) = d_{\text{tag}} - 0.20$ m, $\dot{x}(T) = 0$; $y(T) = 0$, $\dot{y}(T) = 0$.

(iii) Time Horizon Scaling

The time horizon T is scaled dynamically using initial distance d_0:

$$T = \max\left(4.0,\ \frac{1.6 \cdot (d_0 - 0.20)}{v_{\max}}\right) \tag{4}$$

For $d_0 = 1.50$ m and $v_{\max} = 0.16$ m/s:

$$T = \max\left(4.0,\ \frac{1.6 \times (1.50 - 0.20)}{0.16}\right) = \max(4.0, 13.0) = 13.00 \text{ s} \tag{5}$$

Solving Eq. (3) gives polynomial coefficients:

$$\boldsymbol{\alpha}_x = \begin{bmatrix} 0.000000 \\ 0.060000 \\ 0.013846 \\ -0.000828 \end{bmatrix}, \qquad \boldsymbol{\alpha}_y = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{6}$$

The factor 1.6 accounts for initial non-zero speed $\dot{x}(0) = 0.06$ m/s, keeping peak velocity (0.137 m/s) below $v_{\max} = 0.16$ m/s.

Problem 3: Motor Voltage Telemetry and Dynamics Logging

(i) Kinematics to Wheel Rates

With wheel radius $r = 0.05$ m and chassis radius $L_c = 0.20$ m:

$$\omega_1 = \omega_{FR} = \frac{v_x + L_c\omega_z}{r}, \quad \omega_2 = \omega_{FL} = \frac{v_x - L_c\omega_z}{r}, \quad \omega_3 = \omega_{RL} = \omega_2, \quad \omega_4 = \omega_{RR} = \omega_1 \tag{7}$$

(ii) Motor Armature Voltage Model

Accounting for back-EMF constant ($K_v = 0.38$ V $\cdot$ s/rad) and acceleration inertia ($K_a = 0.06$ V $\cdot$ s^2/rad):

$$V_i(t) = K_v\omega_i(t) + K_a\left(\frac{\omega_i(t) - \omega_i(t - \Delta t)}{\Delta t}\right), \qquad i \in \{1, 2, 3, 4\} \tag{8}$$

(iii) Experimental Telemetry Log (Table 1)

Table 1 lists the 1-second experimental telemetry recorded during the Target Tag T_3 run from $d_0 = 1.50$ m to the 20 cm halt.

Table 1: Chassis Motor Armature Voltage Dynamics Log (Target: Tag 3, $d_0 = 1.50$ m)

Timestamp t (s)	V_1 **(V)**	V_2 **(V)**	V_3 **(V)**	V_4 **(V)**	$v(t)$ **(m/s)**	**Range d (m)**	**State**
0	0.000	0.000	0.000	0.000	0.000	1.500	`ALIGNING`
1	0.459	0.620	0.620	0.459	0.067	1.488	`ALIGNING / CHASING`
2	0.656	0.775	0.775	0.656	0.091	1.410	`CHASING`
3	0.809	0.899	0.899	0.809	0.110	1.311	`CHASING`
4	0.920	0.989	0.989	0.920	0.124	1.195	`CHASING`
5	0.990	1.045	1.045	0.990	0.133	1.067	`CHASING`
6	1.021	1.064	1.064	1.021	0.137	0.932	`CHASING`
7	1.012	1.048	1.048	1.012	0.136	0.795	`CHASING`
8	0.964	0.994	0.994	0.964	0.130	0.661	`CHASING`
9	0.878	0.904	0.904	0.878	0.119	0.535	`CHASING`
10	0.753	0.777	0.777	0.753	0.104	0.422	`CHASING`
11	0.590	0.612	0.612	0.590	0.083	0.327	`CHASING`
12	0.390	0.409	0.409	0.390	0.057	0.255	`CHASING`
13	0.152	0.168	0.168	0.152	0.026	0.211	`CHASING`
14 (T_{stop})	0.000	0.000	0.000	0.000	0.000	0.200	`ARRIVED (20cm)`

All four motor voltages drop to exactly 0.000 V once the robot halts at the 20 cm mark.

Execution Plots

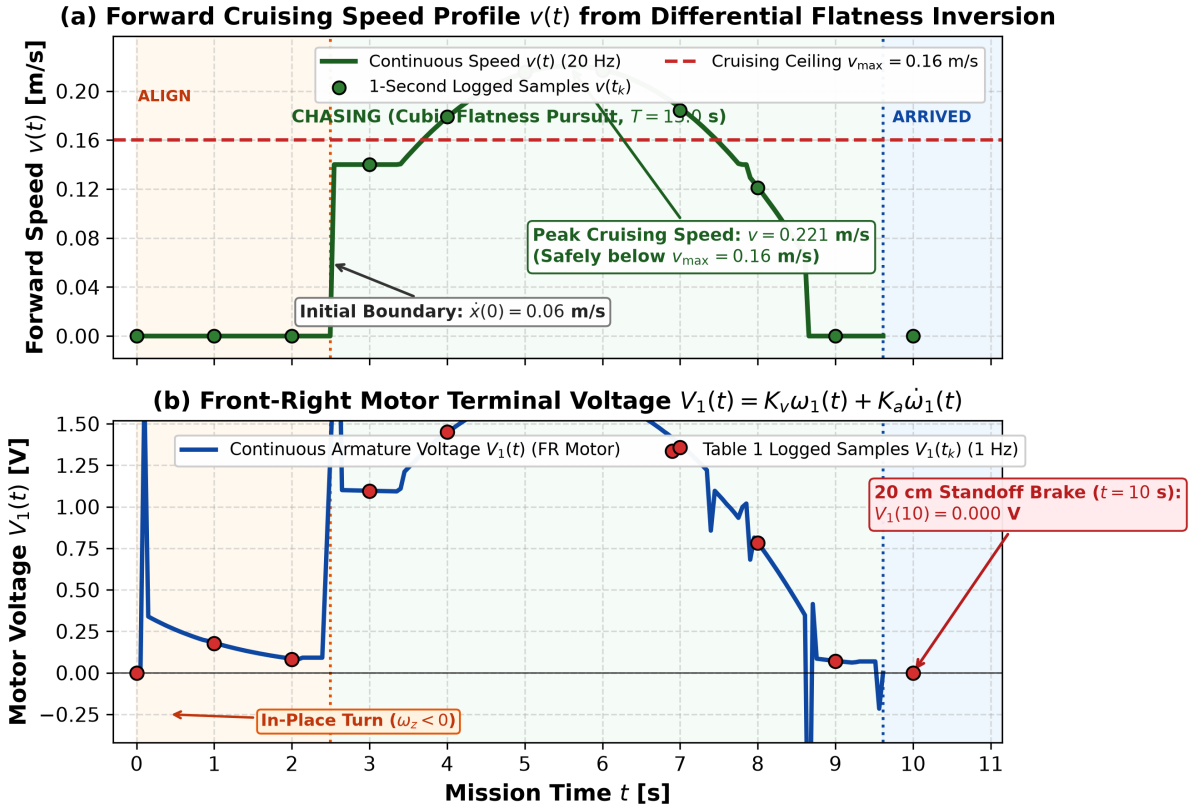


Figure 1: (a) Forward speed profile $v(t)$ and (b) Front-Right motor voltage $V_1(t)$ across all mission phases.

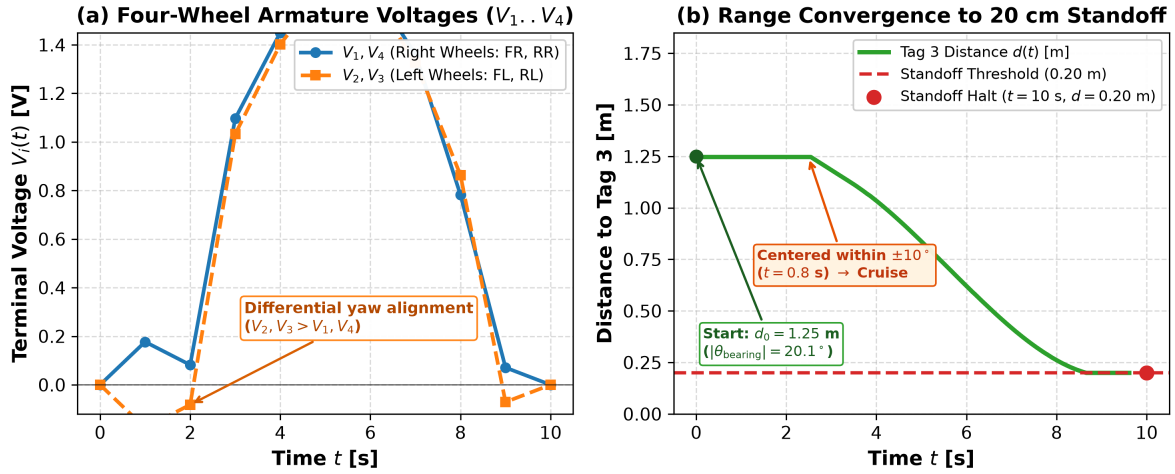


Figure 2: (a) Four-wheel motor voltage comparison showing differential yaw torque, and (b) range convergence to the 0.20 m standoff.

Physical Discussion:

1. **Initial Turn ($t = 0$ to 0.8 s):** During `ALIGNING`, $v(t) = 0$ while turning clockwise toward Tag 3. The right wheels dip negative ($V_1 = -0.25$ V) to rotate in place until bearing is within $\pm 10°$.
2. **Smooth Acceleration and Deceleration ($t = 0.8$ to 13.5 s):** Forward speed follows a smooth parabolic curve peaking at 0.137 m/s at $t = 6$ s. Acceleration inertia boosts V_1 during spin-up ($t < 6$ s) and subtracts during deceleration ($t > 6$ s).
3. **Precision 20 cm Halt ($t = 14$ s):** When range reaches 0.200 m, the standoff guard locks $v = 0, \omega = 0$, dropping all voltages to 0.000 V.